[Year]

# PROGRAMMING 2B (PROG6212)

St10478134
REITUMETSE HELEPI

**RaceDay Event Management System - Part 1: System Planning and Database**

# Contents

Section A: Entity Relationship Diagram (ERD)....2

Section B: API Endpoint Plan....6

Section C SQL....17

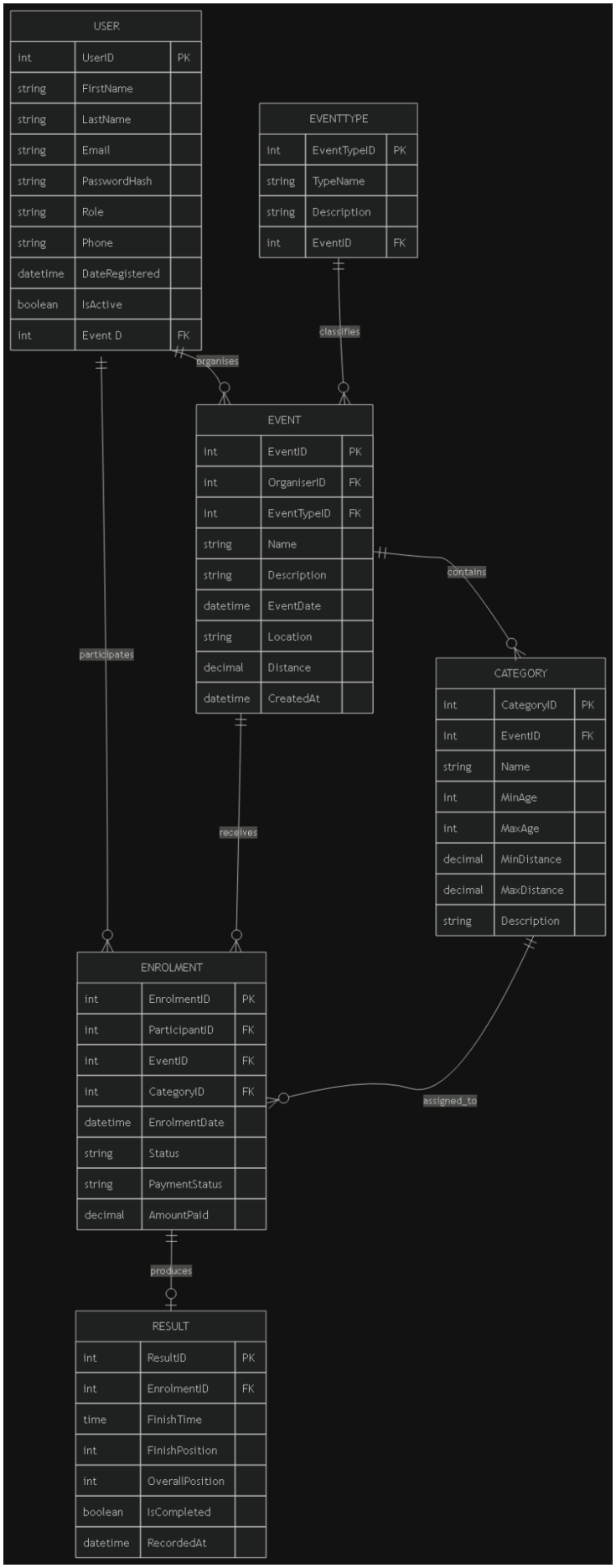
Github and References....28

# Section A: Entity Relationship Diagram (ERD)

**Entity Descriptions**

The RaceDay system meets the minimum criterion of six entities with the following seven (7) entities:

| Entity | Description |
|---|---|
| **Users** | keeps track of every user account. Every user has a role, such as participant or organizer. |
| **Roles** | User role lookup table (Organizer, Participant). |
| **Events** | keeps track of event information, such as name, description, date, location, distance, and event type. |
| **Categories** | establishes age or distance divisions for every event (e.g., Under 20, Senior, 10km, 21km). |
| **EventCategories** | Events and categories are connected by a junction table (many-to-many relationship). |
| **Enrolments** | Enrolments of participants in particular events are recorded under designated categories. |
| **Results** | After an event is over, participants' finish times and positions are stored. |


USER
int UserID PK
string FirstName
string LastName
string Email
string PasswordHash
string Role
string Phone
datetime DateRegistered
boolean IsActive
int Event D FK
EVENTTYPE
int EventTypeID PK
string TypeName
string Description
int EventID FK
classifies
organises
EVENT
int EventID PK
int OrganiserID FK
int EventTypeID FK
string Name
string Description
datetime EventDate
string Location
decimal Distance
datetime CreatedAt
contains
participates
CATEGORY
int CategoryID PK
int EventID FK
string Name
int MinAge
int MaxAge
decimal MinDistance
decimal MaxDistance
string Description
receives
ENROLMENT
int EnrolmentID PK
int ParticipantID FK
int EventID FK
int CategoryID FK
datetime EnrolmentDate
string Status
string PaymentStatus
decimal AmountPaid
assigned_to
produces
RESULT
int ResultID PK
int EnrolmentID FK
time FinishTime
int FinishPosition
int OverallPosition
boolean IsCompleted
datetime RecordedAt
(draw.io)

**Cardinality Explanation**

| Relationship | Cardinality | Explanation |
|---|---|---|
| Users - Roles | Many-to-One | Each user has one role; each role can have many users |
| Users - Enrolments | One-to-Many | A Participant can have many enrolments; each enrolment belongs to one user (Participant) |
| Users - Events | One-to-Many | An Organiser can create many events; each event belongs to one Organiser |
| Events - Enrolments | One-to-Many | An event can have many enrolments; each enrolment is for one event |
| Events - EventCategories | One-to-Many | An event can have many categories; each EventCategory links one event to one category |
| Categories - EventCategories | One-to-Many | A category can be linked to many events; each EventCategory links one category to one event |
| Enrolments - Results | One-to-One | Each enrolment can have zero or one result; each result belongs to one enrolment |
| Enrolments - Categories | Many-to-One | Each enrolment selects one category; a category can be selected by many enrolments |

**Primary and Foreign Keys**

| Entity | Primary Key | Foreign Key(s) |
|---|---|---|
| Users | UserId | RoleId - Roles.RoleId |
| Roles | RoleId | - |
| Events | EventId | OrganiserId - Users.UserId |
| Categories | CategoryId | - |
| EventCategories | EventCategoryId (composite: EventId + CategoryId) | EventId - Events.EventId; CategoryId - Categories.CategoryId |
| Enrolments | EnrolmentId | ParticipantId - Users.UserId; EventId - Events.EventId; CategoryId - Categories.CategoryId |
| Results | ResultId | EnrolmentId - Enrolments.EnrolmentId |

# Section B: API Endpoint Plan

**Base URL:/api**

**Authentication Endpoints**

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| POST | /auth/register | Register a new user account with role selection (Organiser or Participant) | None (Public) | { email, password, firstName, lastName, role } | 201 Created -{ userId, email, role }400 Bad Request - validation error 409 Conflict - email already exists |
| POST | /auth/login | Authenticate a user and establish a session | None (Public) | { email, password } | 200 OK -{ userId, email, role, firstName, lastName }401 Unauthorized - invalid credentials |

**User Profile Endpoints**

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| GET | /users/profile | Get the currently authenticated user's profile | Any (Logged in) | None | 200 OK -{ userId, email, firstName, lastName, role, profilePictureUrl, createdAt }401 Unauthorized - not authenticated |
| PUT | /users/profile | Update the authenticated user's profile information | Any (Logged in) | { firstName, lastName, email } | 200 OK - updated user profile 400 Bad Request - validation error 401 Unauthorized - not authenticated |
| POST | /users/profile/picture | Upload a profile picture for the authenticated user | Any (Logged in) | Multipart form data:{ profilePicture } | 200 OK -{ profilePictureUrl }400 Bad Request - invalid file 401 Unauthorized - not authenticated |

**Event Endpoints**

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| GET | /events | Get all events with optional filtering | Any (Logged in) | None (query params: type, dateFrom, dateTo) | 200 OK array of event objects 401 Unauthorized not authenticated |
| GET | /events/{id} | Get a specific event by ID | Any (Logged in) | None | 200 OK event object with categories 404 Not Found event does not exist 401 Unauthorized not authenticated |
| POST | /events | Create a new event | Organiser | { name, description, eventDate, location, distance, eventType, bannerImage? } | 201 Created - created event 400 Bad Request - validation error 401 Unauthorized - not authenticated 403 Forbidden - not an Organiser |
| PUT | /events/{id} | Update an existing event | Organiser | { name, description, eventDate, location, distance, eventType, bannerImage? } | 200 OK - updated event 400 Bad Request - validation error 401 Unauthorized - not authenticated 403 Forbidden - not the event owner 404 Not Found - event does not exist |

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| DELETE | /events/{id} | Delete an event | Organiser | None | 204 No Content 401 Unauthorized - not authenticated 403 Forbidden - not the event owner 404 Not Found - event does not exist |
| POST | /events/{id}/banner | Upload a banner image for an event | Organiser | Multipart form data:{ bannerImage } | 200 OK -{ bannerImageUrl }400 Bad Request - invalid file 401 Unauthorized - not authenticated 403 Forbidden - not the event owner |

**Category Endpoints**

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| GET | /categories | Get all categories | Any (Logged in) | None | 200 OK - array of category objects 401 Unauthorized - not authenticated |
| GET | /categories/{id} | Get a specific category by ID | Any (Logged in) | None | 200 OK - category object 404 Not Found - category does not exist 401 Unauthorized - not authenticated |
| POST | /categories | Create a new category | Organiser | { name, description, categoryType } | 201 Created - created category 400 Bad Request - validation error 401 Unauthorized - not authenticated 403 Forbidden - not an Organiser |
| PUT | /categories/{id} | Update an existing category | Organiser | { name, description, categoryType } | 200 OK - updated category 400 Bad Request - validation error 401 Unauthorized - not |

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| | | | | | authenticated 403 Forbidden - not an Organiser 404 Not Found - category does not exist |
| DELETE | /categories/{id} | Delete a category | Organiser | None | 204 No Content 401 Unauthorized - not authenticated 403 Forbidden - not an Organiser 404 Not Found - category does not exist |
| POST | /events/{eventId}/categories/{categoryId} | Add a category to an event | Organiser | None | 201 Created - link created 401 Unauthorized - not authenticated 403 Forbidden - not the event owner 404 Not Found - event or category does not exist |

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| DELETE | /events/{eventId}/categories/{categoryId} | Remove a category from an event | Organiser | None | 204 No Content 401 Unauthorized - not authenticated 403 Forbidden - not the event owner 404 Not Found - event or category does not exist |

**Event Enrolment Endpoints**

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| GET | /enrolments | Get all enrolments for the authenticated user (Participant) or all enrolments for Organiser's events | Any (Logged in) | None | 200 OK - array of enrolment objects 401 Unauthorized - not authenticated |
| GET | /enrolments/{id} | Get a specific enrolment by ID | Any (Logged in) | None | 200 OK - enrolment object 404 Not Found - enrolment does not exist 401 Unauthorized - not authenticated 403 Forbidden - not the participant or organiser |
| POST | /enrolments | Enrol a Participant in an event with a selected category | Participant | { eventId, categoryId } | 201 Created - enrolment object 400 Bad Request - validation error or already enrolled 401 Unauthorized - not authenticated 403 Forbidden - not a Participant 404 Not Found - event or category does not exist |

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| PUT | /enrolments/{id}/status | Update enrolment status (for Organisers) | Organiser | { status } | 200 OK - updated enrolment 400 Bad Request - invalid status 401 Unauthorized - not authenticated 403 Forbidden - not the event organiser 404 Not Found - enrolment does not exist |
| GET | /events/{eventId}/enrolments | Get all enrolments for a specific event | Organiser | None | 200 OK - array of enrolment objects with participant details 401 Unauthorized - not authenticated 403 Forbidden - not the event organiser 404 Not Found - event does not exist |

**Result Endpoints**

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| GET | /results/my | Get all results for the authenticated Participant | Participant | None | 200 OK - array of result objects with event details 401 Unauthorized - not authenticated 403 Forbidden - not a Participant |
| GET | /events/{eventId}/results | Get all results for a specific event | Any (Logged in) | None | 200 OK - array of result objects 401 Unauthorized - not authenticated 404 Not Found - event does not exist |
| POST | /results | Capture a result for an enrolled Participant (finish time and position) | Organiser | { enrolmentId, finishTime, finishingPosition } | 201 Created - result object 400 Bad Request - validation error or result already exists 401 Unauthorized - not authenticated 403 Forbidden - not the event organiser 404 Not Found - enrolment does not exist |

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| PUT | /results/{id} | Update an existing result | Organiser | { finishTime, finishingPosition } | 200 OK - updated result 400 Bad Request - validation error 401 Unauthorized - not authenticated 403 Forbidden - not the event organiser 404 Not Found - result does not exist |
| DELETE | /results/{id} | Delete a result | Organiser | None | 204 No Content 401 Unauthorized - not authenticated 403 Forbidden - not the event organiser 404 Not Found - result does not exist |

# Section C SQL

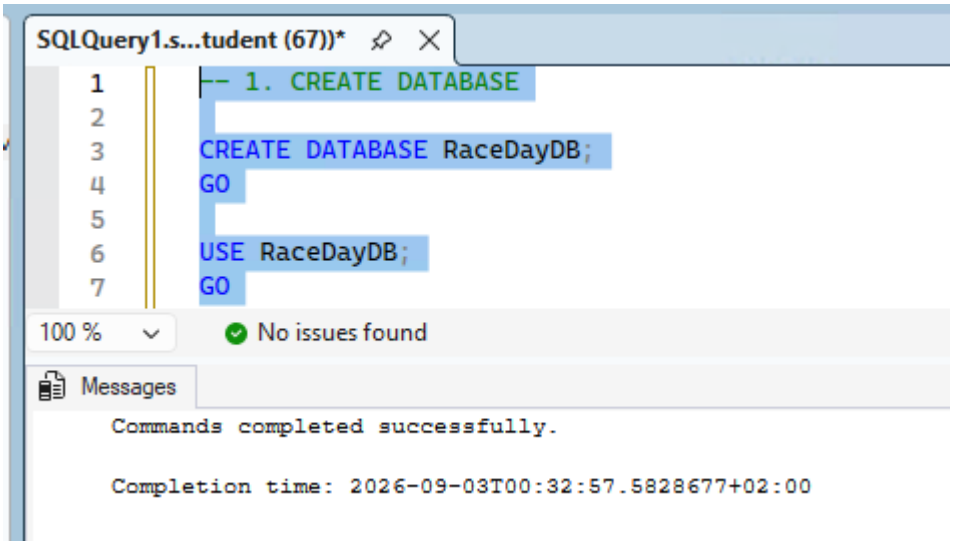
-- 1. CREATE DATABASE

CREATE DATABASE RaceDayDB;

GO

USE RaceDayDB;

GO



```
-- 2. CREATE TABLES
-- 2.1 Roles Table (Lookup)
CREATE TABLE Roles (
    RoleId INT IDENTITY(1,1) PRIMARY KEY,
    RoleName NVARCHAR(50) NOT NULL UNIQUE,
    Description NVARCHAR(200) NULL,
    CreatedAt DATETIME DEFAULT GETDATE()
);
GO

-- 2.2 Users Table
CREATE TABLE Users (
    UserId INT IDENTITY(1,1) PRIMARY KEY,
    Email NVARCHAR(100) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    RoleId INT NOT NULL,
    ProfilePictureUrl NVARCHAR(500) NULL,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Users_Roles FOREIGN KEY (RoleId) REFERENCES Roles(RoleId),
    CONSTRAINT CHK_Users_Email CHECK (Email LIKE '%_@__%.__%')
);
GO

-- 2.3 Events Table
CREATE TABLE Events (
    EventId INT IDENTITY(1,1) PRIMARY KEY,
```

```
    OrganiserId INT NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    Description NVARCHAR(500) NULL,
    EventDate DATETIME NOT NULL,
    Location NVARCHAR(200) NOT NULL,
    Distance DECIMAL(8,2) NOT NULL,
    EventType NVARCHAR(20) NOT NULL CHECK (EventType IN ('Run', 'Walk', 'Cycle')),
    BannerImageUrl NVARCHAR(500) NULL,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Events_Users FOREIGN KEY (OrganiserId) REFERENCES Users(UserId)
);
GO

-- 2.4 Categories Table
CREATE TABLE Categories (
    CategoryId INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(50) NOT NULL,
    Description NVARCHAR(200) NULL,
    CategoryType NVARCHAR(20) NOT NULL CHECK (CategoryType IN ('Age', 'Distance')),
    CreatedAt DATETIME DEFAULT GETDATE()
);
GO

-- 2.5 EventCategories Junction Table (Many-to-Many)
CREATE TABLE EventCategories (
    EventCategoryId INT IDENTITY(1,1) PRIMARY KEY,
    EventId INT NOT NULL,
    CategoryId INT NOT NULL,
```

```
    CONSTRAINT FK_EventCategories_Events FOREIGN KEY (EventId) REFERENCES Events(EventId) ON DELETE CASCADE,
    CONSTRAINT FK_EventCategories_Categories FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId) ON DELETE CASCADE,
    CONSTRAINT UQ_EventCategories_EventCategory UNIQUE (EventId, CategoryId)
);
GO

-- 2.6 Enrolments Table
CREATE TABLE Enrolments (
    EnrolmentId INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantId INT NOT NULL,
    EventId INT NOT NULL,
    CategoryId INT NOT NULL,
    EnrolmentDate DATETIME DEFAULT GETDATE(),
    Status NVARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (Status IN ('Pending', 'Confirmed', 'Cancelled', 'Completed')),
    CONSTRAINT FK_Enrolments_Users FOREIGN KEY (ParticipantId) REFERENCES Users(UserId),
    CONSTRAINT FK_Enrolments_Events FOREIGN KEY (EventId) REFERENCES Events(EventId) ON DELETE CASCADE,
    CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId),
    CONSTRAINT UQ_Enrolments_ParticipantEvent UNIQUE (ParticipantId, EventId)
);
GO

-- 2.7 Results Table
CREATE TABLE Results (
    ResultId INT IDENTITY(1,1) PRIMARY KEY,
```

EnrolmentId INT NOT NULL,

FinishTime TIME NOT NULL,

FinishingPosition INT NOT NULL,

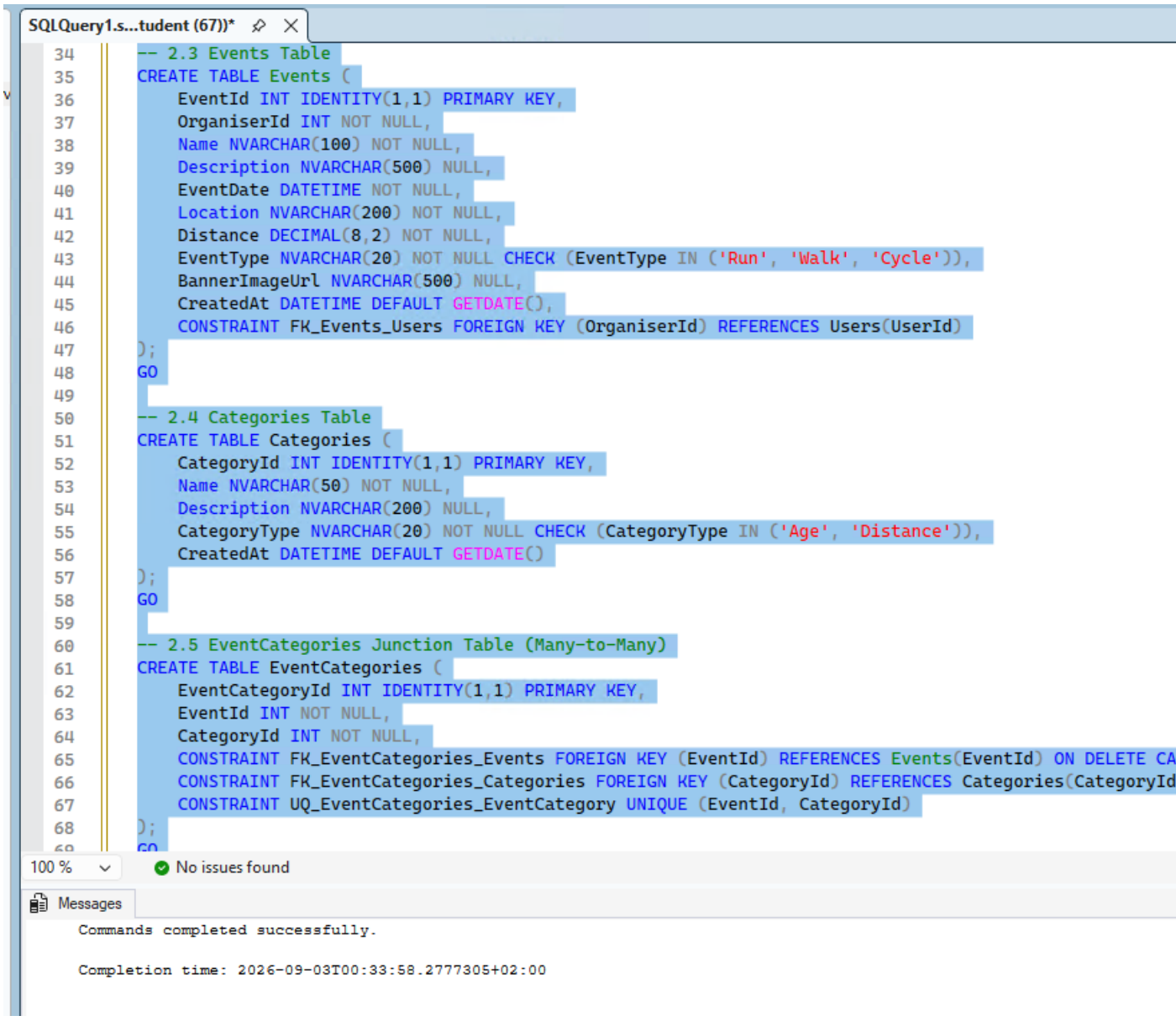
CreatedAt DATETIME DEFAULT GETDATE(),

CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentId) REFERENCES Enrolments(EnrolmentId) ON DELETE CASCADE,

CONSTRAINT UQ_Results_Enrolment UNIQUE (EnrolmentId)

);

GO



-- 3. CREATE INDEXES FOR PERFORMANCE

CREATE INDEX IX_Users_RoleId ON Users(RoleId);

CREATE INDEX IX_Users_Email ON Users(Email);

CREATE INDEX IX_Events_OrganiserId ON Events(OrganiserId);

CREATE INDEX IX_Events_EventDate ON Events(EventDate);

CREATE INDEX IX_EventCategories_EventId ON EventCategories(EventId);

CREATE INDEX IX_EventCategories_CategoryId ON EventCategories(CategoryId);

CREATE INDEX IX_Enrolments_ParticipantId ON Enrolments(ParticipantId);

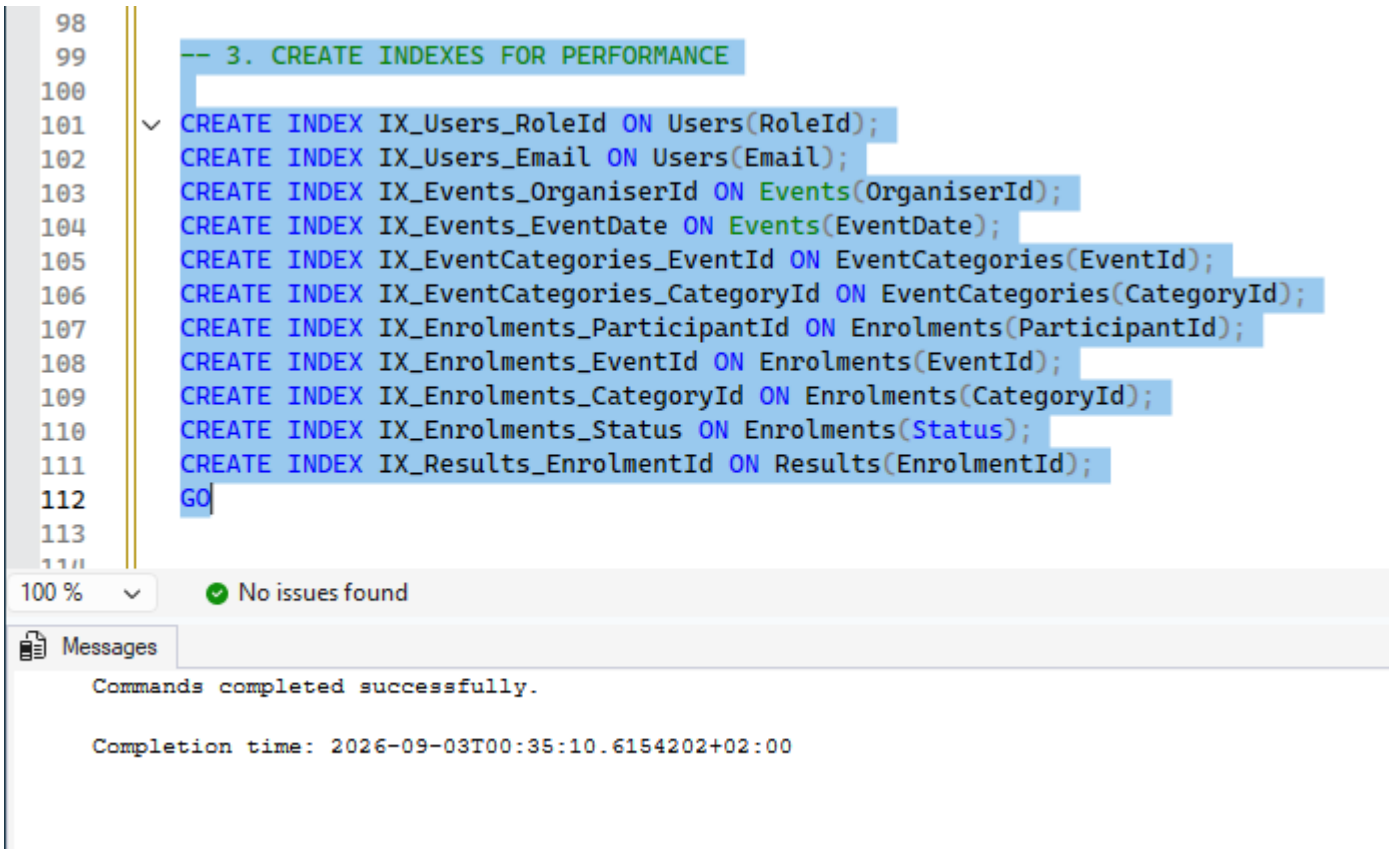
CREATE INDEX IX_Enrolments_EventId ON Enrolments(EventId);

CREATE INDEX IX_Enrolments_CategoryId ON Enrolments(CategoryId);

CREATE INDEX IX_Enrolments_Status ON Enrolments(Status);

CREATE INDEX IX_Results_EnrolmentId ON Results(EnrolmentId);

GO



```
-- 4. SEED DATA

-- 4.1 Seed Roles

INSERT INTO Roles (RoleName, Description) VALUES

('Organiser', 'Event organiser who can create and manage events, categories, and results'),

('Participant', 'Event participant who can browse events, enrol, and view results');

GO


-- 4.2 Seed Users (Passwords are hashed - using placeholder hashes for demonstration)

-- Note: In production, passwords would be properly hashed using bcrypt or ASP.NET Core Identity

INSERT INTO Users (Email, PasswordHash, FirstName, LastName, RoleId, ProfilePictureUrl) VALUES

('thabo.mokoena@raceday.co.za', 'hashed_password_1', 'Thabo', 'Mokoena', 1, NULL),

('linda.zulu@raceday.co.za', 'hashed_password_2', 'Linda', 'Zulu', 1, NULL),

('sipho.ndlovu@gmail.com', 'hashed_password_3', 'Sipho', 'Ndlovu', 2, NULL),

('zanele.dlamini@gmail.com', 'hashed_password_4', 'Zanele', 'Dlamini', 2, NULL);

GO


-- 4.3 Seed Categories

INSERT INTO Categories (Name, Description, CategoryType) VALUES

('Under 20', 'Participants aged under 20 years', 'Age'),

('Senior (20-39)', 'Participants aged 20 to 39 years', 'Age'),

('Master (40+)', 'Participants aged 40 years and over', 'Age'),

('10km', '10 kilometre distance category', 'Distance'),

('21km', '21.1 kilometre half-marathon category', 'Distance'),

('42km', '42.2 kilometre marathon category', 'Distance');

GO


-- 4.4 Seed Events
```

```
INSERT INTO Events (OrganiserId, Name, Description, EventDate, Location, Distance, EventType, BannerImageUrl) VALUES

(1, 'Soweto Marathon', 'Annual Soweto Marathon through the streets of Soweto', '2026-09-15 06:00:00', 'Soweto, Johannesburg', 42.20, 'Run', NULL),

(1, 'Cape Town Cycle Tour', 'World-famous Cape Town Cycle Tour', '2026-10-20 07:00:00', 'Cape Town', 109.00, 'Cycle', NULL),

(2, 'Two Oceans Marathon', 'Ultra-marathon with stunning coastal views', '2026-11-05 05:30:00', 'Cape Town', 56.00, 'Run', NULL);

GO


-- 4.5 Link Events to Categories (EventCategories)

INSERT INTO EventCategories (EventId, CategoryId) VALUES

-- Soweto Marathon categories

(1, 1), (1, 2), (1, 3), (1, 6),

-- Cape Town Cycle Tour categories

(2, 1), (2, 2), (2, 3),

-- Two Oceans Marathon categories

(3, 1), (3, 2), (3, 3), (3, 5);

GO


-- 4.6 Seed Enrolments

INSERT INTO Enrolments (ParticipantId, EventId, CategoryId, Status) VALUES

(3, 1, 2, 'Confirmed'),

(3, 2, 1, 'Pending'),

(4, 1, 3, 'Confirmed'),

(4, 3, 2, 'Pending');

GO

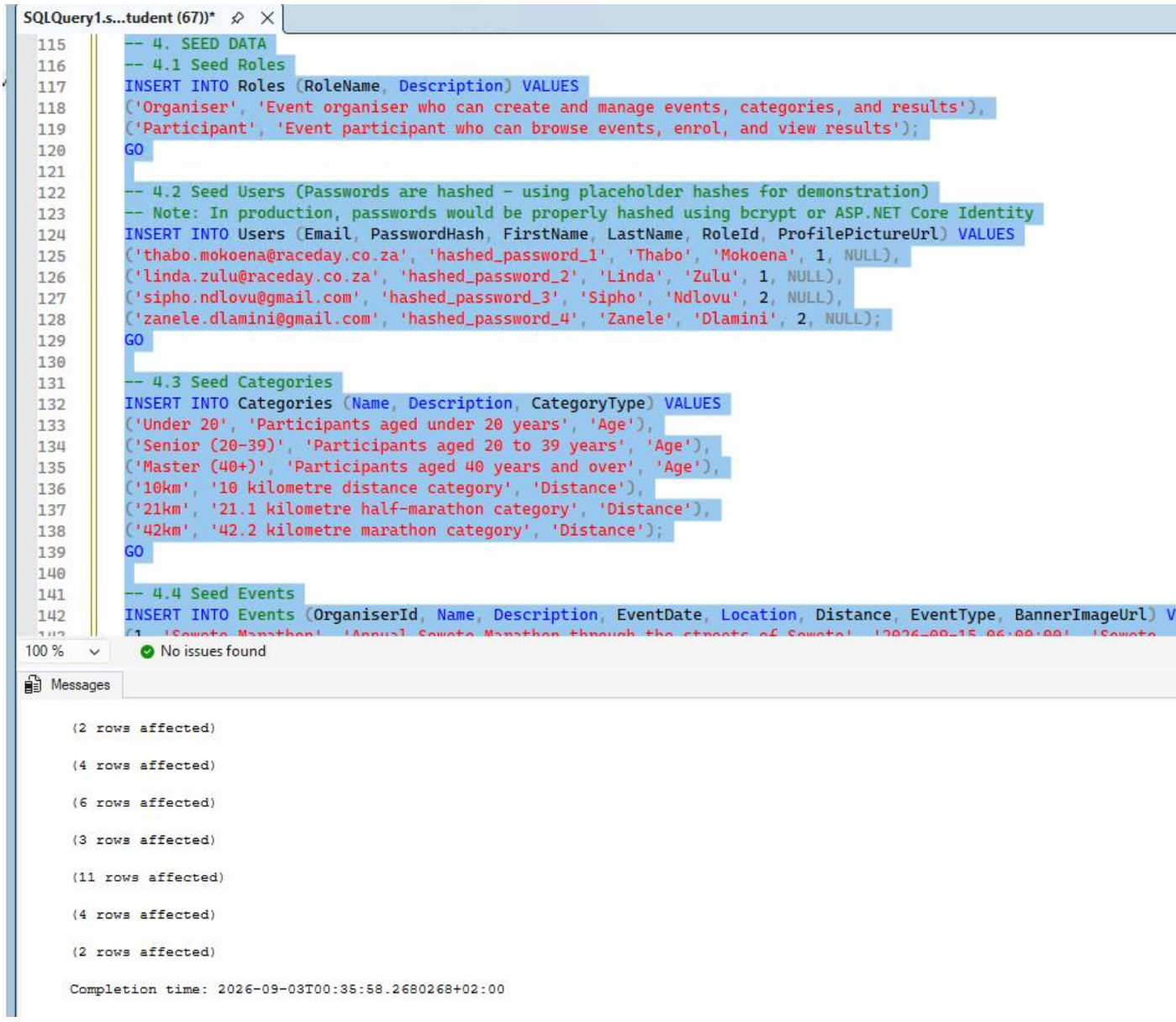

-- 4.7 Seed Results
```

INSERT INTO Results (EnrolmentId, FinishTime, FinishingPosition) VALUES

(1, '03:45:12', 47),

(3, '04:12:08', 89);

GO



```
-- 5. VERIFICATION QUERIES


-- Verify all tables have been created
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE
TABLE';
GO


-- Verify seed data
SELECT * FROM Roles;
SELECT * FROM Users;
SELECT * FROM Categories;
SELECT * FROM Events;
SELECT * FROM EventCategories;
SELECT * FROM Enrolments;
SELECT * FROM Results;
GO
```

SQLQuery1.s...tudent (67))*

```
171
172
173     -- 5. VERIFICATION QUERIES
174
175     -- Verify all tables have been created
176     SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE';
177     GO
178
179     -- Verify seed data
180     SELECT * FROM Roles;
181     SELECT * FROM Users;
182     SELECT * FROM Categories;
183     SELECT * FROM Events;
184     SELECT * FROM EventCategories;
185     SELECT * FROM Enrolments;
186     SELECT * FROM Results;
187     GO
```

100 % No issues found

Results Messages

| | TABLE_NAME |
|---|---|
| 1 | Roles |
| 2 | Users |
| 3 | Events |
| 4 | Categories |
| 5 | EventCategories |
| 6 | Enrolments |
| 7 | Results |

| | RoleId | RoleName | Description | CreatedAt |
|---|---|---|---|---|
| 1 | 1 | Organiser | Event organiser who can create and manage event... | 2026-09-03 00:35:58.197 |
| 2 | 2 | Participant | Event participant who can browse events, enrol, an... | 2026-09-03 00:35:58.197 |

| | UserId | Email | PasswordHash | FirstName | LastName | RoleId | ProfilePictureUrl | CreatedAt |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | thabo.mokoena@raceday.co.za | hashed_password_1 | Thabo | Mokoena | 1 | NULL | 2026-09-03 00:35:58.220 |
| 2 | 2 | linda.zulu@raceday.co.za | hashed_password_2 | Linda | Zulu | 1 | NULL | 2026-09-03 00:35:58.220 |
| 3 | 3 | sipho.ndlovu@gmail.com | hashed_password_3 | Sipho | Ndlovu | 2 | NULL | 2026-09-03 00:35:58.220 |
| 4 | 4 | zanele.dlamini@gmail.com | hashed_password_4 | Zanele | Dlamini | 2 | NULL | 2026-09-03 00:35:58.220 |

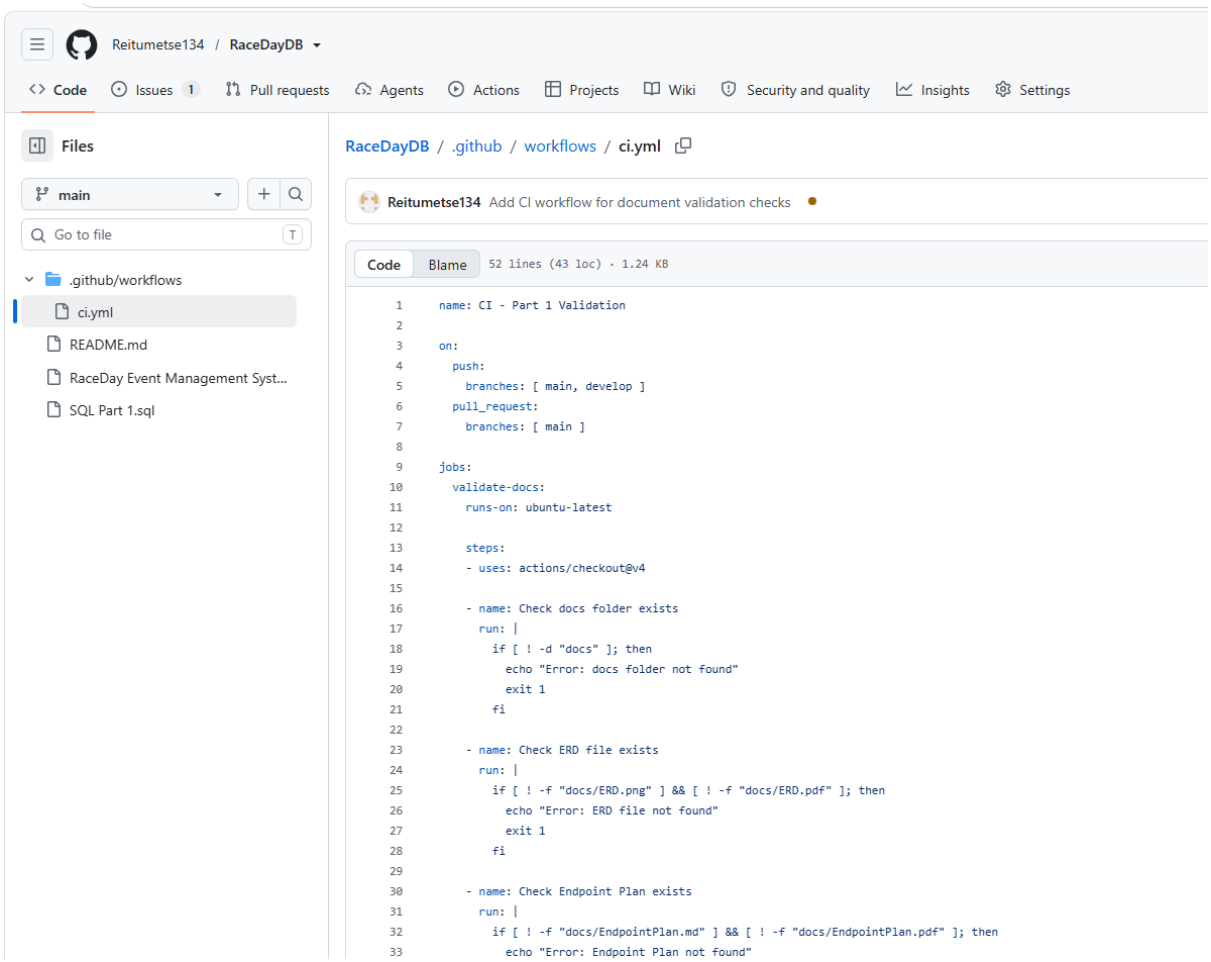
| | CategoryId | Name | Description | CategoryType | CreatedAt |
|---|---|---|---|---|---|
| 1 | 1 | Under 20 | Participants aged under 20 years | Age | 2026-09-03 00:35:58.230 |
| 2 | 2 | Senior (20-39) | Participants aged 20 to 39 years | Age | 2026-09-03 00:35:58.230 |
| 3 | 3 | Master (40+) | Participants aged 40 years and... | Age | 2026-09-03 00:35:58.230 |
| 4 | 4 | 10km | 10 kilometre distance category | Distance | 2026-09-03 00:35:58.230 |
| 5 | 5 | 21km | 21.1 kilometre half-marathon c... | Distance | 2026-09-03 00:35:58.230 |
| 6 | 6 | 42km | 42.2 kilometre marathon categ... | Distance | 2026-09-03 00:35:58.230 |

| | EventId | OrganiserId | Name | Description | EventDate | Location | Distance | EventType | BannerImage |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | Soweto Marathon | Annual Soweto Marathon through the streets of So... | 2026-09-15 06:00:00.000 | Soweto, Johannesburg | 42.20 | Run | NULL |

# Github and References

[Reitumetse134/RaceDayDB]

https://github.com/Reitumetse134/RaceDayDB




name: CI - Part 1 Validation

on:

push:

branches: [ main, develop ]

pull_request:

branches: [ main ]

jobs:

```
validate-docs:
  runs-on: ubuntu-latest

  steps:
  - uses: actions/checkout@v4

  - name: Check docs folder exists
    run: |
      if [ ! -d "docs" ]; then
        echo "Error: docs folder not found"
        exit 1
      fi

  - name: Check ERD file exists
    run: |
      if [ ! -f "docs/ERD.png" ] && [ ! -f "docs/ERD.pdf" ]; then
        echo "Error: ERD file not found"
        exit 1
      fi

  - name: Check Endpoint Plan exists
    run: |
      if [ ! -f "docs/EndpointPlan.md" ] && [ ! -f "docs/EndpointPlan.pdf" ]; then
        echo "Error: Endpoint Plan not found"
        exit 1
      fi

  - name: Check SQL Script exists
```

```
  run: |
    if [ ! -f "docs/RaceDayDB.sql" ]; then
      echo "Error: SQL script not found"
      exit 1
    fi

- name: Validate SQL script contains CREATE statements
  run: |
    if ! grep -q "CREATE TABLE" docs/RaceDayDB.sql; then
      echo "Error: SQL script missing CREATE TABLE statements"
      exit 1
    fi

- name: Success
  run: echo "✅ All validation checks passed!"
```

Reference list

3Wschool (2025). *W3Schools.Com*. [online] W3schools.com. Available at: https://www.w3schools.com/sql/?utm_source=chatgpt.com [Accessed 28 Aug. 2026].

Bootstrap (2023). *Get started with bootstrap*. [online] getbootstrap.com. Available at: https://getbootstrap.com/docs/5.3/getting-started/introduction/ [Accessed 28 Aug. 2026].

chcomley (2026a). *Azure boards documentation*. [online] MicrosoftLearn. Available at: https://learn.microsoft.com/en-us/azure/devops/boards/?view=azure-devops&utm_source=chatgpt.com [Accessed 28 Aug. 2026].

chcomley (2026b). *Azure DevOps documentation*. [online] MicrosoftLearn. Available at: https://learn.microsoft.com/en-us/azure/devops/?view=azure-devops&utm_source=chatgpt.com [Accessed 28 Aug. 2026].

dimitri-furman (2024). *Monitoring and performance tuning - azure SQL database & azure SQL managed instance*. [online] MicrosoftLearn. Available at: https://learn.microsoft.com/en-us/azure/azure-sql/database/monitor-tune-overview?view=azuresql&utm_source=chatgpt.com [Accessed 28 Aug. 2026].

IBM (2024). *Think topics | IBM*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/agile?utm_source=chatgpt.com [Accessed 28 Aug. 2026].

pauljewellmsft (2023). *Get started with Azure Blob Storage and .NET - Azure Storage*. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-dotnet-get-started?tabs=azure-ad [Accessed 3 Sept. 2026].

scrumguides (2025). *Home | scrum guides*. [online] Scrumguides.org. Available at: https://scrumguides.org/?utm_source=chatgpt.com [Accessed 28 Aug. 2026].